

Memoria proyecto práctico 2026.

Fernando Fernández Martínez.

Leonardo Chavez Tarazona.

Álvaro Pérez Romero.

1. Contexto del proyecto.....	3
2. Diagrama de clases.....	4
2.1. Dominio principal.....	5
2.2. Entidades JPA y mapeo.....	6
2.3. Puertos de repositorio e implementaciones.....	7
2.4. Servicios y controladores.....	8
3. Decisiones técnicas.....	9
4. Métricas.....	11
4.1. Cobertura de pruebas.....	12
4.2. Complejidad ciclomática.....	13
4.3. Code Smells.....	16
4.4. Duplicación de código.....	17
5. Demo funcional.....	18

1. Contexto del proyecto.

Una organización presta servicios técnicos a clientes (mantenimiento informático, soporte, reparaciones, etc.).

Actualmente gestiona las solicitudes de servicio de manera informal y desea disponer de un sistema que permita:

- Registrar solicitudes.
- Asignarlas a técnicos.
- Controlar su estado.
- Evaluar tiempos de resolución.
- Obtener métricas básicas.

El sistema deberá diseñarse desde el inicio con criterios de mantenibilidad y calidad.

Adicionalmente a las peticiones iniciales se nos pidió poder reabrir solicitudes.

2. Diagrama de clases.

Este proyecto sigue la arquitectura hexagonal y se divide en:

- Dominio: Contiene el modelo de negocio.
- Entidades de dominio: Contiene las entidades JPA que se mapean en la base de datos, permiten hacer la conversión entre entidades y dominio.
- Infraestructura interfaces: Define los puertos/repositorios abstractos, declara cómo se accede a los datos pero sin amarrarse a tecnologías concretas.
- Infraestructura: Contiene las implementaciones concretas de la persistencia.
- Servicio: Contiene la lógica de negocio y usa los repositorios.
- Api: Contiene los controladores y expone los servicios.

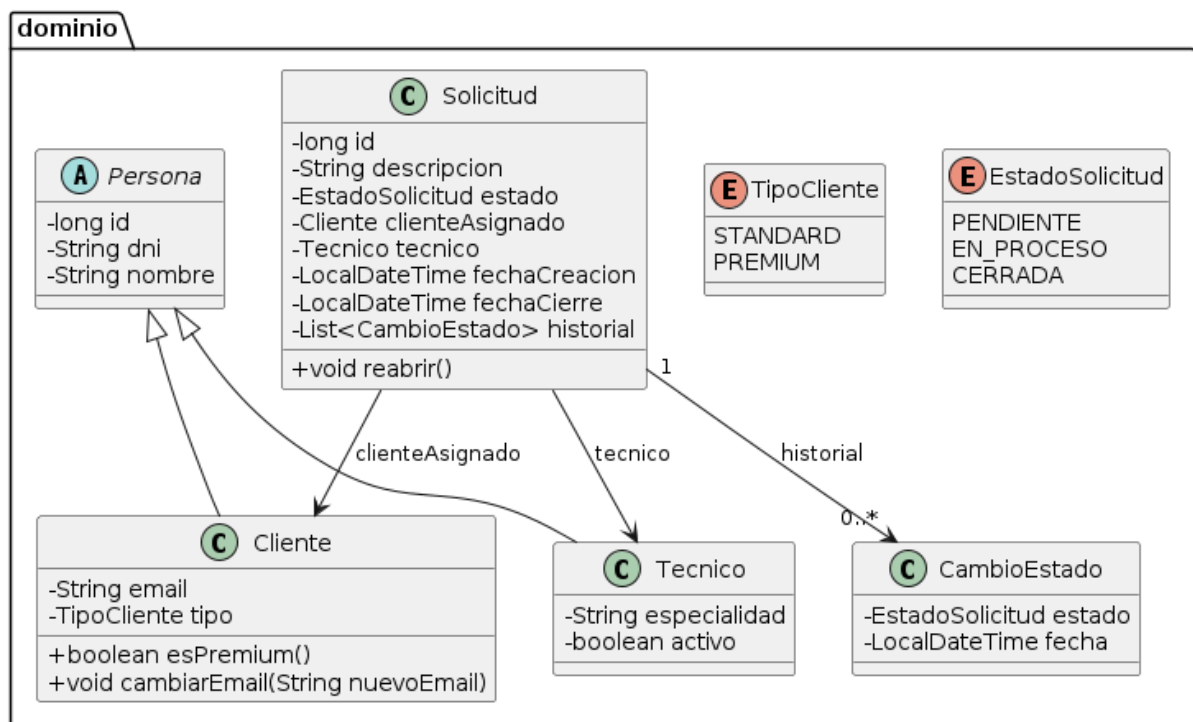
A continuación se presenta el diagrama de clases del sistema. Debido al número de clases que lo componen, su visualización al completo es poco práctica y difícil de interpretar. Por este motivo, se ha optado por dividirlo en varios fragmentos, permitiendo analizar cada sección con mayor claridad y facilitando la comprensión de la estructura y las relaciones existentes entre los distintos componentes del sistema.

2.1. Dominio principal.

En la siguiente figura se muestra el diagrama de clases final correspondiente al dominio de la aplicación.

En él puede observarse la clase **Persona**, de la que heredan las clases **Técnico** y **Cliente**. Ambas entidades comparten varios atributos y comportamientos comunes. Mediante el uso de herencia se evita la duplicación de código, se mejora la mantenibilidad y se favorece la reutilización de funcionalidades compartidas.

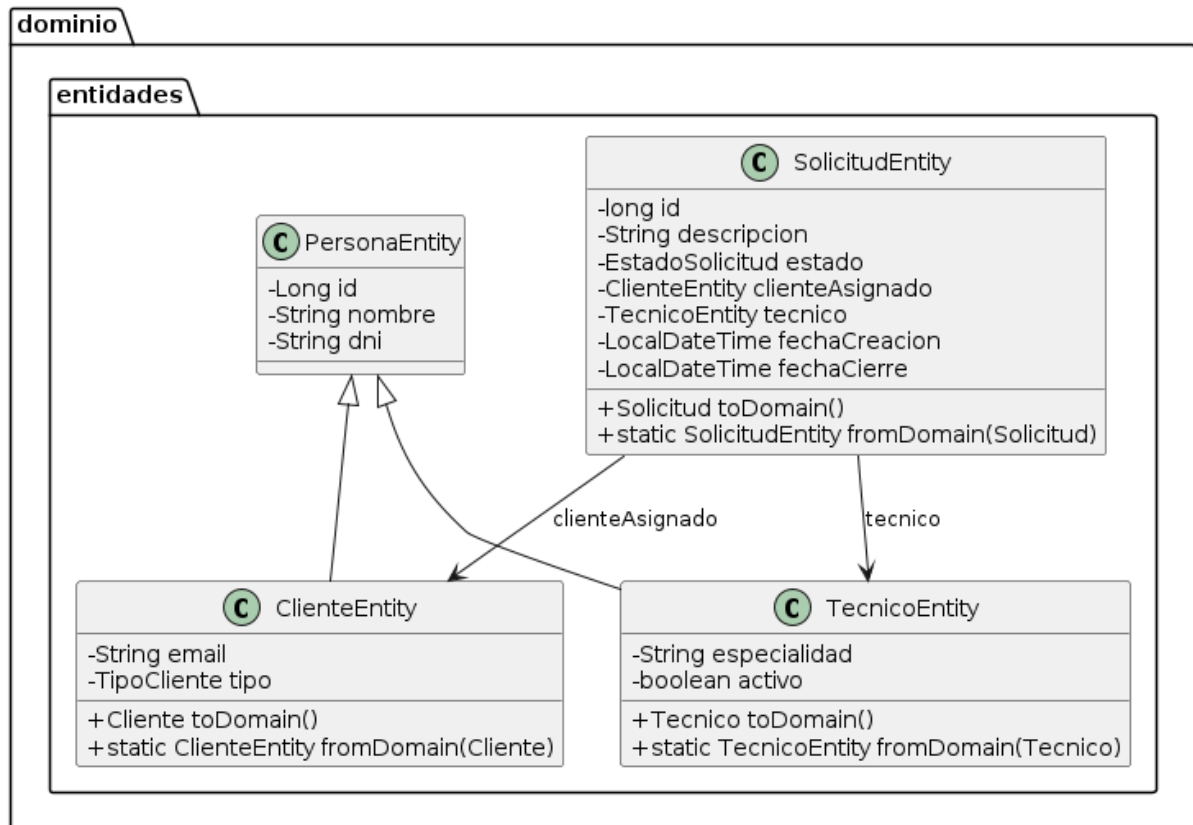
Además de estas clases, el dominio incluye la entidad **Solicitud**, que representa el elemento central del sistema y sobre la que se articulan los principales procesos de negocio. Finalmente, se han definido dos enumeraciones que permiten representar de forma controlada determinados valores del dominio, mejorando la legibilidad del código y reduciendo la posibilidad de errores derivados del uso de valores arbitrarios.



2.2. Entidades JPA y mapeo.

En esta figura se muestran las entidades utilizadas en la capa de persistencia. Constituyen una representación del modelo de dominio adaptado a JPA.

Como se explicará más adelante , se opta por mantener separadas las clases del dominio y las entidades de persistencia, de este modo, las entidades actúan como una traducción del modelo de dominio a una estructura compatible con JPA, facilitando tanto la persistencia de los datos como la evolución independiente de ambas capas.



2.3. Puertos de repositorio e implementaciones.

La aplicación puede operar utilizando dos mecanismos de almacenamiento diferentes: persistencia en memoria y persistencia mediante JPA.

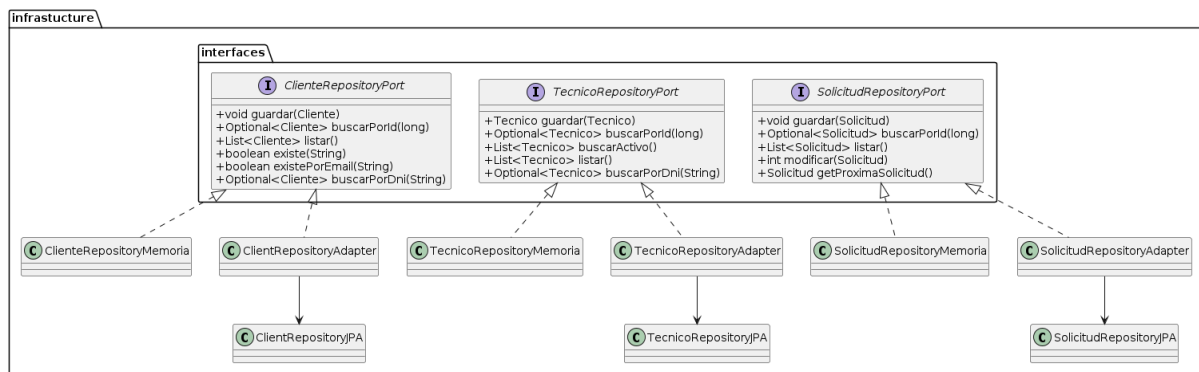
Durante las primeras etapas del desarrollo, el sistema trabajaba exclusivamente con datos almacenados en memoria. Cuando se incorporó la persistencia real, se decidió conservar la implementación original para demostrar que en nuestro sistema pueden coexistir distintas tecnologías de acceso a datos bajo una misma abstracción.

La selección del mecanismo de persistencia se realiza mediante los perfiles de Spring. En el fichero `src/main/resources/application.properties` se encuentra la propiedad:

```
1 spring.application.name=mgsc
2 spring.profiles.active=jpa
```

Modificando el valor de `spring.profiles.active` entre `memory` y `jpa`, es posible elegir cómo funciona la aplicación.

Gracias a que ambas implementaciones respetan las mismas interfaces el sistema permanece inalterado a pesar del cambio.

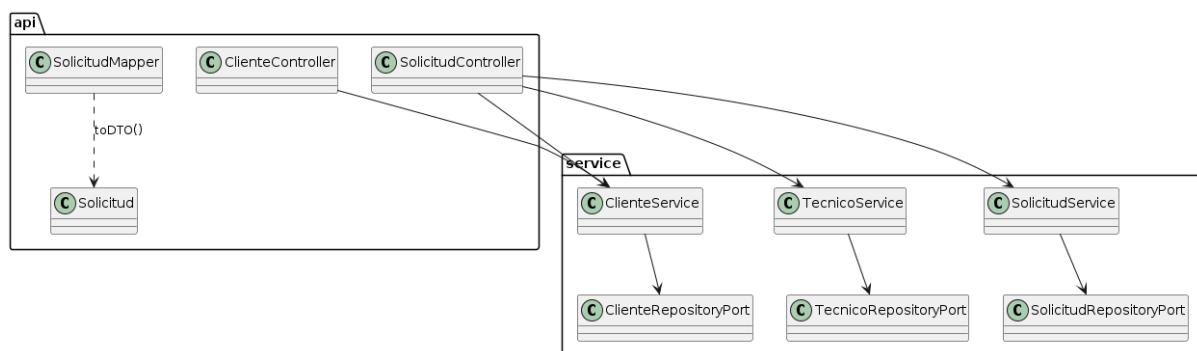


2.4. Servicios y controladores.

La capa de servicios implementa la lógica de negocio de la aplicación, en ella se encuentran las reglas y procesos que definen el comportamiento del sistema, y mantiene separada la responsabilidad.

Los servicios actúan como intermediarios entre los controladores y los distintos puertos de acceso a datos. De este modo, los controladores delegan en los servicios la ejecución de las operaciones de negocio, mientras que los servicios utilizan los puertos para interactuar con los repositorios sin depender de una tecnología. en concreto.

Esta organización favorece la separación de responsabilidades, reduce el acoplamiento entre capas y facilita tanto el mantenimiento como la evolución futura de la aplicación.



3. Decisiones técnicas.

Las primeras decisiones del proyecto se tomaron durante la definición del dominio. En particular, las clases Cliente y Solicitud contienen atributos con valores limitados y enumerables, por lo que se buscó evitar el uso de valores “hardcodeados”. Para ello, se optó por la creación de enumeraciones, permitiendo definir de manera clara y controlada todos los posibles valores que dichos atributos pueden adoptar.

Además, el proyecto debía ajustarse a una arquitectura hexagonal, caracterizada por la existencia de un núcleo de negocio aislado y distintos adaptadores encargados de comunicarse con él. Siguiendo este enfoque, la aplicación se estructuró en tres capas principales: Controladores, Servicios y Repositorios.

Siguiendo el enfoque de arquitectura hexagonal, los controladores no contienen lógica de negocio propia, sino que funcionan como adaptadores entre el usuario o cliente externo y el núcleo de la aplicación.

Los servicios son los encargados de implementar la lógica de negocio de nuestra aplicación y de comunicar el controlador con los repositorios.

Los repositorios, por su parte, implementan una interfaz común, lo que permite intercambiar distintas implementaciones según las necesidades de ejecución del proyecto sin afectar al resto de la aplicación. De esta forma, se favorece la flexibilidad, la mantenibilidad y el desacoplamiento entre las distintas partes del sistema.

Una de las primeras decisiones importantes que tomamos surgió al implementar persistencia real en la aplicación. En un primer momento, se planteó adaptar todas las clases del dominio para que pudieran persistir mediante JPA, esta no era una solución óptima y se optó por crear un conjunto de clases específicas para la persistencia. Esta solución nos permite mantener la modularidad deseada y facilita la realización de pequeñas modificaciones en la información persistente, si fuera necesario, sin afectar al modelo de dominio.

Para ello, implementamos una entidad JPA para cada clase del dominio, incorporando mecanismos de conversión que permiten transformar objetos del dominio en entidades persistentes y viceversa.

Esta decisión dio lugar a uno de los primeros problemas relevantes del proyecto. Hasta ese momento, las identificaciones (ID) de los objetos se asignan manualmente, pero JPA gestiona este proceso de forma automática. Por ello, decidimos delegar la responsabilidad de asignar los identificadores en los repositorios. Como consecuencia, los objetos no dispondrán de un ID válido hasta ser almacenados.

1. Crear entidad en el dominio
↳ id = -1 (sin asignar)
↓
2. Enviar al repositorio para guardar.
↓
3. Repositorio detecta id == -1
↳ Asigna nuevo ID
↓
4. Entidad guardada con ID permanente

Además de integrarse mejor con JPA, esta decisión aporta una ventaja adicional: si en el futuro se incorpora un nuevo repositorio basado en una tecnología diferente, el mecanismo de generación y asignación de identificadores podrá adaptarse a dicha tecnología sin afectar al resto del sistema.

La siguiente decisión relevante también estuvo relacionada con la capa de persistencia, concretamente durante la implementación de los repositorios. Era necesario mantener las interfaces ya definidas para los repositorios y, al mismo tiempo, realizar la conversión entre los objetos del dominio y sus correspondientes entidades JPA.

Para resolver este problema, implementamos una serie de adaptadores que actúan como puente entre ambas representaciones. Estos adaptadores se encargan de transformar los objetos del dominio en entidades persistentes y viceversa además de asegurar que se cumple la interfaz de los repositorios, permitiendo que la lógica de negocio permanezca completamente desacoplada de la tecnología de persistencia utilizada.

Gracias a esta solución, conseguimos preservar el aislamiento entre el modelo de dominio y JPA, reduciendo el acoplamiento del sistema y facilitando futuras modificaciones o sustituciones de la capa de persistencia.

La siguiente decisión de diseño estuvo relacionada con los controladores que serían los encargados de atender y responder a las peticiones HTTP de la aplicación.

Para evitar la exposición directa del modelo de dominio, creamos clases específicas para la capa de presentación. Así, mantenemos el desacoplamiento, evitando que cambios internos en el dominio afecten directamente a los contratos expuestos al exterior.

Esta aproximación nos proporciona una gran flexibilidad a la hora de definir la información intercambiada con los clientes de la aplicación. En particular, permite añadir, eliminar o modificar atributos expuestos de forma sencilla, sin necesidad de alterar el modelo de dominio.

Además, esta separación de responsabilidades contribuye a mantener una API REST más clara, estable y adaptada a las necesidades de los consumidores, facilitando tanto su evolución como su mantenimiento a largo plazo.

4. Métricas.

A continuación, se expondrán y analizarán algunas de las métricas más relevantes proporcionadas por SonarQube que nos permiten evaluar distintos aspectos de la calidad del software, como la mantenibilidad, la fiabilidad, la seguridad o la cobertura de pruebas.

Su análisis resulta de gran utilidad para identificar posibles problemas en el código, detectar áreas susceptibles de mejora y comprobar si el proyecto cumple con los estándares de calidad establecidos.

4.1. Cobertura de pruebas.

En este proyecto hay 128 tests distribuidos en 19 clases de test.

Estos tests se organizan en categorías:

- Tests de Dominio: ClienteTest, SolicitudTest, TecnicoTest
- Tests de Servicios: ClienteServiceTest, SolicitudServiceTest, TecnicoServiceTest
- Tests de API/Controllers: ClienteControllerTest, SolicitudControllerWebMvcTest, TecnicoControllerTest, y variantes con WebMvc
- Tests de Infraestructura/Repositorios: ClienteRepositoryMemoriaTest, SolicitudRepositoryMemoriaTest, TecnicoRepositoryMemoriaTest, y sus variantes Adapter
- Tests de Integración: SolicitudJpaTest

Los tests están bien estructurados cubriendo las diferentes capas de la aplicación (dominio, servicio, controladores e infraestructura).

Con estos test nuestra aplicación supera sobradamente el +80% de cobertura necesaria.



En esta gráfica podemos ver el estado de la cobertura desde que empezó a registrarse. La línea más oscura representa las líneas a cubrir, y la más oscura las líneas que los test cubre.

El porcentaje de cobertura del proyecto se representa con la separación de estas líneas, a menos separación mayor cobertura.

Durante todo el proyecto la cobertura se mantiene en torno al 90%, incluso en torno al 17 de Mayo donde podemos ver una separación mayor, provocado por el aumento de número de líneas de código.

4.2. Complejidad ciclomática.

En la tabla podremos ver la complejidad ciclomática de cada clase de este proyecto, todas las clases están enlazada al proyecto de cloud, para poder revisarlas si se desea.

La complejidad ciclomática del proyecto se mantiene en valores moderados para la mayoría de las clases, lo que indica una estructura relativamente sencilla y fácil de mantener.

Las capas de DTOs, entidades e interfaces presentan valores bajos (entre 0 y 8), reflejando que su función principal es el transporte o almacenamiento de datos sin lógica de negocio compleja.

Sin embargo, se observan mayores niveles de complejidad en las clases relacionadas con la gestión de solicitudes, como [SolicitudController.java](#), [Solicitud.java](#), [ClienteService.java](#) y [SolicitudRepositoryMemoria.java](#), que alcanzan valores entre 17 y 18.

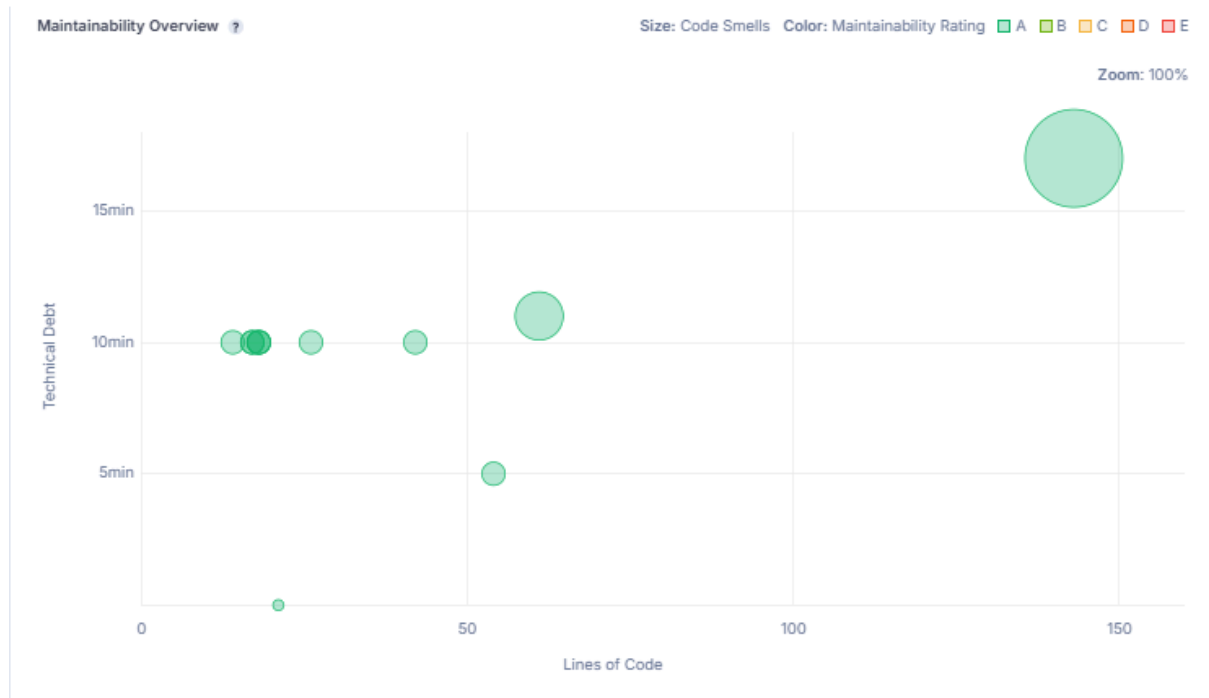
Esto muestra que la mayor parte de la lógica de decisión de la aplicación se concentra en el proceso de gestión de solicitudes, y aunque, estos valores no son excesivamente altos, podrían revisarse estas clases para evaluar posibles refactorizaciones que mejoren la mantenibilidad y reduzcan la complejidad del código.

RUTA			Complejidad ciclomática.
src/main/java/com/example/mgsc/api/			98
	ClienteController.java		7
	SolicitudController.java		18
	SolicitudMapper.java		3
	TecnicoController.java		8
	DTOs/		62
		ClienteRequestDTO.java	6
		ClienteResponseDTO.java	6
		PersonaRequestDTO.java	6
		PersonaResponseDTO.java	8
		SolicitudRequestDTO.java	6
		SolicitudResponseDTO.java	18
		TecnicoRequestDTO.java	6
		TecnicoResponseDTO.java	6
src/main/java/com/example/mgsc/dominio			58
	CambioEstado.java		3
	Cliente.java		8
	EstadoSolicitud.java		0
	Persona.java		7
	Solicitud.java		18
	Tecnico.java		6
	TipoCliente.java		0
	/entidades		16
		ClienteEntity.java	3
		PersonaEntity.java	3
		SolicitudEntity.java	7
		TecnicoEntity.java	3
src/main/java/com/example/mgsc/infrastructure			61
	ClienteRepositoryMemoria.java		12
	ClientRepositoryAdapter.java		7
	ClientRepositoryJPA.java		0
	SolicitudRepositoryAdapter.java		10
	SolicitudRepositoryJPA.java		0
	SolicitudRepositoryMemoria.java		17
	TecnicoRepositoryAdapter.java		6
	TecnicoRepositoryJPA.java		0
	TecnicoRepositoryMemoria.java		9
	/interfaces		0
		ClienteRepositoryPort.java	0

RUTA			Complejidad ciclomática.
		TecnicoRepositoryPort.java	0
		SolicitudRepositoryPort.java	0
src/main/java/com/example/mgsc/service			37
	ClienteService.java		10
	SolicitudService.java		17
	TecnicoService.java		10
MgscApplication.java			1

4.3. Code Smells.

En la siguiente diagrama de burbujas podemos ver las clases de nuestro proyecto que contienen “código que huele”, cada burbuja representa una clase, el tamaño de las mismas indica el número de code smells, su posición vertical indica el tiempo de solucionarlo, su posición horizontal el número de líneas de la clase y por último el color de las burbujas indica la gravedad.



De primeras podemos ver que todas las burbujas tienen colores verdes, lo cual indica una buena mantenibilidad.

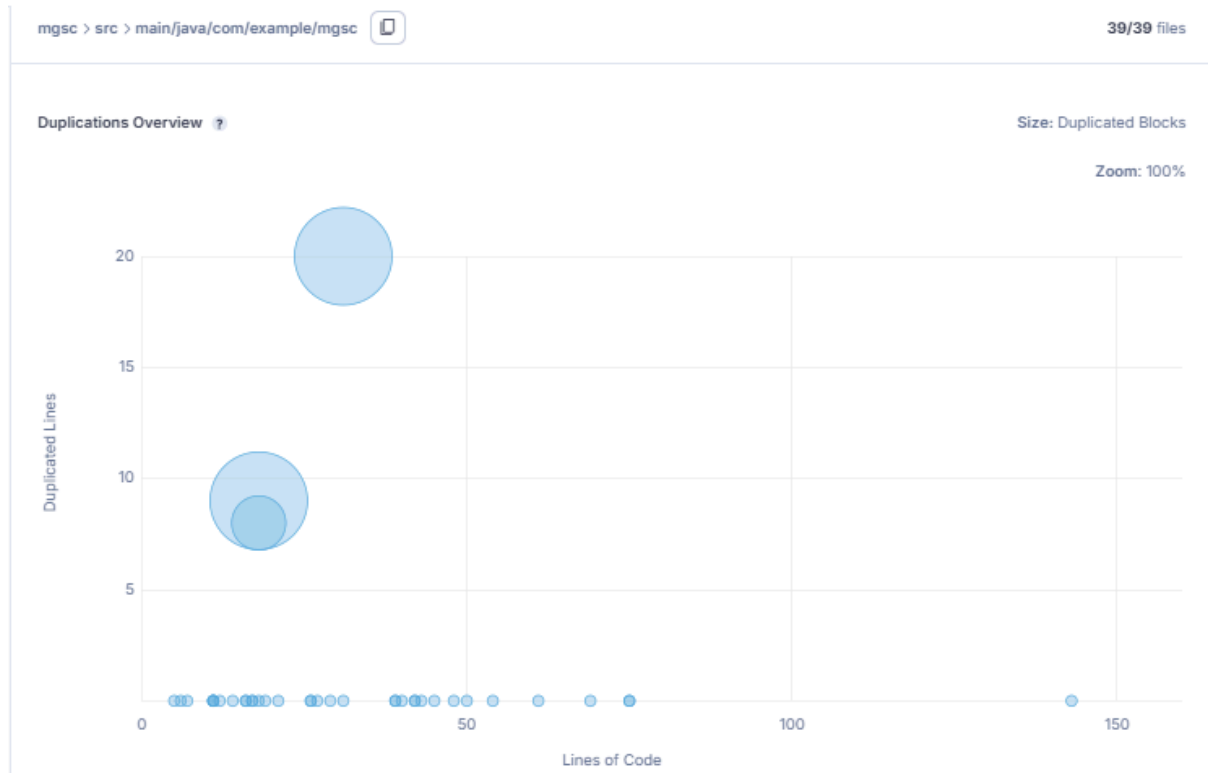
En segundo lugar llama la atención la burbuja grande de arriba a la derecha, la cual corresponde a [SolicitudController.java](#), a pesar de ser la de mayor tamaño no supone una gran deuda técnica.

La mayoría de las clases tienen menos de 50 líneas y un tiempo estimado para arreglarlo de unos 10 minutos, por lo que la deuda técnica actualmente está controlada.

Si se desea ver con más detalle el informe de Sonar la imagen está enlazada a su fuente.

4.4. Duplicación de código.

A continuación podemos ver otro gráfico de burbujas, donde el eje vertical indica el número de líneas duplicadas, el eje horizontal el número de líneas de la clase y el tamaño de las burbujas el tamaño de bloques duplicados.



Lo que más llama la atención son esas tres burbujas donde dos se encuentran superpuestas, la burbuja más arriba corresponde a [Cliente.java](#), las dos restantes, la mayor corresponde a [ClienteRequestDTO.java](#) y la de menor tamaño a [ClienteResponseDTO.java](#). Podemos ver que todas están relacionadas con la parte de Cliente de nuestra aplicación y probablemente sea más una estadística inflada que un problema real, sino más bien una concentración de similitudes en las clases relacionadas con Cliente y Técnico. Esto probablemente se deba a la presencia de constructores, getters y setters con estructuras semejantes, un tipo de duplicación habitual y generalmente aceptable en modelos de dominio y DTOs.

Además, la ausencia de duplicación en el resto de las clases sugiere que la base de código mantiene un buen nivel de separación, por lo que estas métricas deben interpretarse con cautela y en contexto, sin asumir automáticamente la existencia de un problema real.

5. Demo funcional.

La imagen de la demo del proyecto puede lanzarse en [docker hub](#) y debe utilizarse usando swagger o lanzando el docker directamente desde el proyecto.